



El Dashboard del Desarrollador

Taller Práctico de Maquetación Avanzada

Módulo	3 — Maquetación Avanzada
Nivel	Intermedio-Avanzado (HTML5 + CSS3)
Duración estimada	8 – 12 horas (individual/equipo)
Prerequisitos	Flexbox · Grid · HTML Semántico
Siguiente módulo	JavaScript DOM + Fetch API

1. Contexto del Proyecto — Historia de Usuario

💡 Nota para el instructor: Lee esta historia con tu grupo antes de comenzar a programar. El contexto narrativo es clave para que los aprendices entiendan por qué cada decisión técnica importa.

El Escenario

La startup TechLink está lanzando su plataforma de gestión de proyectos para pequeñas agencias de software en Latinoamérica. Su equipo de tres desarrolladores full-stack lleva semanas coordinando tareas a través de hojas de cálculo de Google Sheets, hilos de WhatsApp y correos electrónicos. El resultado ha sido el caos.

La CTO, Valentina Ríos, ha definido el MVP (Producto Mínimo Viable): necesitan un dashboard web que les permita ver, crear, editar y eliminar tareas de forma centralizada. El producto se llama SocioTask.

Historia de Usuario Principal

"Como desarrolladora del equipo de TechLink, quiero un dashboard en mi navegador donde pueda ver todas las tareas pendientes organizadas por prioridad, con sus fechas límite y el responsable asignado, para que pueda tomar decisiones rápidas sobre qué atender primero cada mañana."

Tu rol en este taller

Eres el desarrollador front-end contratado por TechLink. Tu entregable de esta semana es la maqueta HTML/CSS del dashboard. En el siguiente sprint, otro desarrollador tomará tu maqueta y le agregará la funcionalidad con JavaScript. Por eso, la calidad de tu código de presentación es crítica: el código que escribas hoy determinará qué tan fácil o difícil será agregar JavaScript mañana.

Criterios de aceptación

- El layout debe funcionar en pantallas de escritorio (1280px+) y móvil (360px+)
- Las tarjetas de tareas deben ser visualmente claras: prioridad, responsable y fecha límite deben identificarse en menos de 3 segundos
- El dashboard debe tener un modo oscuro y un modo claro
- Cada botón de acción (Editar, Completar, Eliminar) debe tener un atributo data-id para ser capturable por JavaScript
- El código CSS debe seguir la metodología BEM para facilitar el mantenimiento

2. Instrucciones Paso a Paso

Paso 0 — Estructura de Archivos del Proyecto

Antes de escribir una sola línea de código, crea esta estructura de carpetas en tu computadora:

```
sociotask/
├── index.html
└── css/
    └── style.css
```

Por ahora usaremos un solo archivo HTML y un solo archivo CSS. En módulos avanzados aprenderás a separar los componentes. ¡La claridad de estructura es profesionalismo desde el día uno!

Paso 1 — Estructura Semántica del index.html

Abre index.html en tu editor y escribe el esqueleto semántico. La regla de oro: cada etiqueta HTML debe describir qué es el contenido, no cómo se ve.

Etiquetas semánticas que usarás

Etiqueta HTML	Rol en el Dashboard	Por qué es semántica
<body>	Contenedor raíz del dashboard	Define el documento entero
<aside>	Sidebar de navegación lateral	Contenido complementario al principal
<header>	Barra superior del dashboard	Cabecera de la página
<main>	Área de contenido principal	Contenido central único por página
<nav>	Menú de navegación	Lista de enlaces de navegación
<section>	Grupos de contenido (Stats, Tareas)	Sección temática con heading
<article>	Cada tarjeta de tarea	Contenido autocontenido e independiente
<h1>–<h3>	Jerarquía de títulos	Estructura de información para SEO y a11y
<button>	Editar, Completar, Eliminar	Elemento interactivo nativo

Estructura HTML base del Dashboard

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SocioTask - Dashboard</title>
  <link rel="stylesheet" href="css/style.css" />
</head>
<body>

  <div class="dashboard">

    <aside class="sidebar">
      <!-- Logo, navegación, perfil de usuario -->
    </aside>

    <header class="header">
      <!-- Título de página + búsqueda + acciones -->
    </header>

    <main class="main">
      <section class="stats"><!-- Estadísticas --></section>
      <section class="tasks"><!-- Tarjetas de tareas --></section>
      <section class="data-map"><!-- Mapa de datos --></section>
    </main>

  </div>
</body>
</html>
```

Paso 2 — Variables CSS y Modo Claro/Oscuro

En tu archivo `css/style.css`, lo primero que debes definir son las Custom Properties (variables CSS). Esta decisión define toda la paleta de colores del proyecto y hace posible el modo oscuro con una sola línea de JavaScript más adelante.

Regla de oro: Nunca escribas un color, fuente o tamaño "hardcodeado" directamente en una propiedad CSS. Siempre usa una variable. Esto te ahorrará horas de refactoring.

Cómo funcionan las Custom Properties

```
:root {
```

```
/* Paleta de colores — Modo Claro (default) */
--color-bg:      #F0F2F8; /* Fondo general */
--color-surface:  #FFFFFF; /* Tarjetas y paneles */
--color-border:   #D1D8EE; /* Bordes sutiles */
--color-text:     #1A1E2E; /* Texto principal */
--color-muted:    #6B758E; /* Texto secundario */
--color-accent:   #4F6EF7; /* Color de marca */

/* Prioridades */
--color-alta:     #EF4444;
--color-media:    #F59E0B;
--color-baja:     #10B981;

/* Dimensiones del layout */
--sidebar-width:  240px;
--header-height:  64px;
}

/* Modo oscuro: solo sobreescribimos los colores */
[data-theme="dark"] {
  --color-bg:      #0F1117;
  --color-surface:  #1A1E2E;
  --color-border:   #2E3455;
  --color-text:     #E8ECF6;
  --color-muted:    #8892AA;
}
```

¿Cómo se activa el modo oscuro? Con JavaScript, cuando el usuario haga clic en el toggle, agregarás `data-theme="dark"` al elemento `<html>` y el CSS hará el resto automáticamente. ¡Sin JavaScript extra!

Paso 3 — Layout con CSS Grid (Santa Claus Layout)

El "Holy Grail" o "Santa Claus Layout" es un patrón clásico de diseño web que tiene: un header superior fijo, un footer inferior (opcional), un área de contenido principal y columnas laterales (sidebars). Nuestro dashboard usa una variante simplificada.

Diagrama del Grid

SIDEBAR	HEADER
sticky full height	MAIN CONTENT (overflow-y: auto — tiene scroll)

CSS del Grid principal

```
.dashboard {
  display: grid;
  grid-template-columns: var(--sidebar-width) 1fr;
  grid-template-rows: var(--header-height) 1fr;
  grid-template-areas:
    "sidebar header"
    "sidebar main";
  min-height: 100vh;
}

.sidebar { grid-area: sidebar; position: sticky; top: 0; height: 100vh; }
.header { grid-area: header; position: sticky; top: 0; z-index: 90; }
.main { grid-area: main; overflow-y: auto; padding: 28px; }
```

¿Por qué grid-template-areas? Porque te permite "nombrar" zonas del layout con palabras legibles. Es autoDocumentado: alguien que lea tu CSS entiende la estructura sin ver el HTML.

Paso 4 — Metodología BEM para CSS

BEM (Block Element Modifier) es una convención de nomenclatura para clases CSS que hace el código predecible, reutilizable y fácil de capturar desde JavaScript.

La fórmula BEM

```
/* Bloque: componente independiente */
.task-card { }

/* Elemento: parte del bloque (doble guión bajo) */
.task-card__title { }
.task-card__actions { }
.task-card__btn { }

/* Modificador: variante del bloque/elemento (doble guión) */
.task-card--completed { } /* tarjeta completada */
.task-card--priority-alta { } /* borde rojo */
.task-card__btn--eliminar { } /* botón rojo al hover */
.task-card__btn--completar { } /* botón verde al hover */
```

Clase BEM	Rol	Por qué facilita JavaScript
.task-card	Bloque: la tarjeta completa	document.querySelectorAll(".task-card")
.task-card__btn	Elemento: cualquier botón de la tarjeta	Selector base para delegar eventos
.task-card__btn--eliminar	Modificador: botón de eliminar	Identificar el tipo de acción al hacer clic
.task-card--completed	Modificador: tarea completada	Alternar visualmente el estado con classList
.task-card--priority-alta	Modificador: prioridad alta	Filtrar tarjetas por prioridad

Paso 5 — Tarjetas de Tareas con Flexbox

Cada tarjeta de tarea es un componente que combina múltiples elementos en una fila horizontal. Flexbox es la herramienta ideal porque necesitamos alineación en un solo eje.

Estructura HTML de una tarjeta (con data-id)

```
<article class="task-card task-card--priority-alta" data-task-id="001">

  <!-- Checkbox de completar -->
  <div class="task-card__checkbox" role="checkbox" aria-checked="false"></div>

  <!-- Contenido principal -->
  <div class="task-card__body">
    <div class="task-card__header">
      <h3 class="task-card__title">Diseñar arquitectura de BD</h3>
      <span class="task-card__badge task-card__badge--alta">Alta</span>
    </div>
    <p class="task-card__desc">Definir esquema de tablas...</p>
    <div class="task-card__meta">
      <span class="task-card__meta-item">📅 Vence: 06 mar 2026</span>
      <span class="task-card__tag">backend</span>
    </div>
  </div>

  <!-- Botones de acción -->
  <div class="task-card__actions">
    <button class="task-card__btn task-card__btn--completar"
      data-id="001">✓</button>
    <button class="task-card__btn task-card__btn--editar">
```

```

        data-id="001">✎</button>
        <button class="task-card__btn task-card__btn--eliminar"
            data-id="001">🗑️</button>
    </div>

</article>

```

Atención crítica: El atributo data-id en cada botón es lo que conectará tu maqueta con JavaScript. Cuando el usuario haga clic en "Eliminar", el event listener leerá el data-id para saber qué tarea borrar del array de datos. ¡Sin ese atributo, el JS tendrá que "adivinar"!

CSS Flexbox de la tarjeta

```

.task-card {
    display: flex;           /* Fila horizontal */
    align-items: flex-start;
    gap: 16px;
    padding: 18px 20px;
    background: var(--color-surface);
    border: 1px solid var(--color-border);
    border-radius: 12px;
}

.task-card__body {
    flex: 1; /* Ocupa todo el espacio disponible */
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.task-card__actions {
    display: flex;
    gap: 6px;
    flex-shrink: 0; /* No se comprime con el contenido */
}

```

Paso 6 — Sección de Estadísticas con Barras de Progreso

Las estadísticas dan una vista rápida del estado del proyecto. Usaremos tarjetas con barras de progreso maquetadas con HTML/CSS puro.

```

<section class="stats">
    <article class="stats__card">

```



```
<div class="stats__card-header">
  <span class="stats__label">Completadas</span>
  <div class="stats__icon stats__icon--green">✔</div>
</div>
<div class="stats__value">18</div>
<div class="stats__progress-bar">
  <!-- El width en % se cambiará con JavaScript -->
  <div class="stats__progress-fill" style="width: 75%"></div>
</div>
</article>
</section>
```

```
.stats {
  display: flex;
  gap: 16px;
  flex-wrap: wrap; /* Baja de línea en pantallas pequeñas */
}

.stats__card { flex: 1; min-width: 180px; }

.stats__progress-bar {
  height: 6px;
  background: var(--color-bg);
  border-radius: 99px;
  overflow: hidden;
}

.stats__progress-fill {
  height: 100%;
  background: var(--color-accent);
  border-radius: 99px;
  transition: width 1s ease;
}
```

Paso 7 — Responsive: Sidebar a Menú Inferior

En móvil, el sidebar lateral ocupa demasiado espacio horizontal. La solución es convertirlo en una barra de navegación inferior, como las apps móviles nativas.

```
@media (max-width: 768px) {

  /* Redefine el Grid: todo en una columna */
  .dashboard {
```

```
grid-template-columns: 1fr;
grid-template-rows: var(--header-height) 1fr auto;
grid-template-areas:
  "header"
  "main"
  "sidebar"; /* El sidebar va AL FINAL */
}

/* Sidebar: fijo en la parte inferior */
.sidebar {
  position: fixed;
  bottom: 0; left: 0; right: 0;
  height: auto;
  flex-direction: row;
}

/* Ocultar elementos que no caben en mobile */
.sidebar__brand,
.sidebar__footer { display: none; }

/* Navegación horizontal */
.sidebar__nav {
  flex-direction: row;
  justify-content: space-around;
  padding: 8px;
}

/* Padding extra para que el main no quede detrás del nav */
.main { padding-bottom: 90px; }
}
```

3. Mapa de Datos — Sustituto del UML

Antes de programar, los desarrolladores diseñan la estructura de sus datos. Esta sección te enseña a pensar como un arquitecto de software: ¿qué información necesita existir para que la aplicación funcione?

Instrucción: Completa este análisis ANTES de escribir cualquier código. Discútelo con tu equipo. Este "Mapa de Datos" será la base del objeto JavaScript que usarás en el próximo módulo.

Entidad Principal: Tarea (Task)

Una "entidad" es un tipo de objeto que necesita guardarse en una base de datos. Piensa en la entidad Tarea: ¿qué información necesitas recordar sobre cada tarea?

Propiedad	Tipo de dato	Obligatorio	Descripción	Ejemplo
id	String (UUID)	Sí (auto)	Identificador único irrepetible	"a1b2-c3d4-..."
titulo	String	Sí	Nombre corto de la tarea	"Diseñar BD"
descripcion	Text	No	Detalle ampliado de la tarea	"Definir tablas..."
fecha_limite	Date	No	Fecha de vencimiento (ISO 8601)	"2026-03-12"
estado	Enum	Sí	Estado actual de la tarea	"pendiente"
prioridad	Enum	Sí	Nivel de urgencia	"alta"
asignado_a	String (FK)	No	ID del usuario responsable	"usr_001"
etiquetas	String[]	No	Tags de categorización	["backend","BD"]
creado_en	Timestamp	Sí (auto)	Fecha de creación automática	"2026-03-04T..."
actualizado_en	Timestamp	Sí (auto)	Última modificación automática	"2026-03-04T..."

Valores posibles para los Enums

Campo	Valores posibles	Representación visual en el dashboard
estado	pendiente en_progreso completada vencida	Color de la tarjeta / badge de estado
prioridad	alta media baja	Borde de color + badge de prioridad

Vista previa: El objeto tarea en JavaScript

En el próximo módulo, cada tarea que veas en el dashboard habrá sido generada desde un objeto como este:

```
const tarea = {
  id: "alb2c3d4-e5f6-7890-abcd-ef1234567890",
  titulo: "Diseñar arquitectura de base de datos",
  descripcion: "Definir esquema de tablas y relaciones...",
  fecha_limite: "2026-03-06",
  estado: "pendiente",
  prioridad: "alta",
  asignado_a: "usr_001",
  etiquetas: ["backend", "BD"],
  creado_en: "2026-03-04T09:00:00Z",
  actualizado_en: "2026-03-04T09:00:00Z"
};
```

Pregunta de reflexión para el equipo: ¿Qué propiedad del objeto "tarea" determina qué clase CSS modifier se aplica a la tarjeta? Pista: task-card--priority-alta.

4. Rúbrica de Evaluación

El puntaje total del taller es de 100 puntos. Cada categoría evalúa un aspecto diferente del código entregado.

Categoría	Peso	Puntos
Semántica HTML5	20%	20 pts
Grid & Flexbox Layout	40%	40 pts
Naming BEM & data-id	20%	20 pts
Mapa de Datos	20%	20 pts
TOTAL	100%	100 pts

Categoría 1 — Semántica HTML5 (20 pts)

Puntos	Descriptor
18–20	Todas las etiquetas son semánticas y correctas. Atributos ARIA presentes en elementos interactivos. Sin divitis. lang="es" definido.
14–17	La mayoría de etiquetas son semánticas. Algún <div> donde debería haber un elemento semántico. Sin atributos ARIA.
10–13	Mezcla de etiquetas semánticas y genéricas. Jerarquía de headings incorrecta (ej: h3 sin h2 previo).
5–9	Uso mínimo de semántica. La estructura está basada en <div> y sin etiquetas descriptivas.
0–4	Sin uso de etiquetas semánticas. Todo el layout está en <div>. Ilegible para screen readers.

Categoría 2 — Grid & Flexbox Layout (40 pts)

Puntos	Descriptor
36–40	Grid implementado correctamente con grid-template-areas. Sidebar sticky funcional. Flexbox en tarjetas, stats y sidebar. Media Query convierte sidebar en bottom nav en mobile. Variables CSS en todo el layout.
28–35	Grid funcional pero sin grid-template-areas (usa columnas directas). Flexbox aplicado. Responsive incompleto: sidebar se oculta en mobile pero no se convierte en bottom nav.

Puntos	Descriptor
18–27	Grid implementado pero con errores (overlay de elementos, áreas mal definidas). Flexbox básico. Sin responsive.
8–17	Intento de Grid pero con fallback a floats o position absolute. Sin Flexbox correctamente aplicado.
0–7	Sin uso de Grid. Sin Flexbox. Layout roto o usando tablas HTML para maquetar.

Categoría 3 — Naming BEM & Atributos data-* (20 pts)

Puntos	Descriptor
18–20	BEM consistente en todo el proyecto. Bloques, elementos y modificadores bien separados. TODOS los botones de acción tienen data-id con valor único. Clases específicas por tipo de acción (--completar, --editar, --eliminar).
14–17	BEM aplicado en la mayoría de componentes con alguna inconsistencia (ej: mezclar camelCase con BEM). data-id presente en botones principales.
10–13	BEM aplicado parcialmente. Algunos componentes usan BEM, otros usan nombres genéricos. data-id presente pero sin clases de tipo de acción.
5–9	BEM mencionado pero mal implementado (triple guión, doble en bloques, etc.). data-id ausente o con valores incorrectos.
0–4	Sin BEM. Clases descriptivas sin metodología. Sin atributos data-id.

Categoría 4 — Mapa de Datos (20 pts)

Puntos	Descriptor
18–20	El Mapa de Datos identifica correctamente las 8–10 propiedades de la entidad Tarea. Cada propiedad tiene nombre, tipo de dato, si es obligatorio y descripción. Los enums tienen sus valores posibles definidos. Se incluye la vista previa del objeto JS.
14–17	Se identifican 6–7 propiedades con sus tipos. Falta alguna propiedad importante (ej: timestamps). Los enums están definidos pero sin todos los valores.
10–13	Se identifican 4–5 propiedades básicas (id, titulo, estado, prioridad). Tipos de dato presentes pero sin descripción. Sin enums definidos.
5–9	Se identifican 2–3 propiedades genéricas. Sin tipos de dato. Mapa incompleto o confuso.
0–4	Mapa de datos ausente o entregado en blanco. Menos de 2 propiedades identificadas.

5. Bonus Track — Tooltips Solo con CSS

★ Este reto es opcional pero suma 10 puntos adicionales. Si lo completas, tu puntaje puede llegar a 110/100.

Un tooltip es el pequeño cuadro de texto que aparece al hacer hover sobre un elemento. Normalmente se implementan con JavaScript, pero con CSS moderno podemos hacerlos funcionar sin una sola línea de JS.

¿Cómo funciona la magia?

El truco usa tres características de CSS que ya conoces:

- El pseudo-elemento `::after` para crear el cuadro del tooltip
- La función `attr()` para leer el contenido del atributo `data-tooltip` del HTML
- La pseudo-clase `:hover` para mostrar el tooltip solo cuando el cursor está encima

Implementación paso a paso

Paso A: Agrega el atributo `data-tooltip` al HTML

```
<button class="task-card__btn task-card__btn--completar"
        data-id="001"
        data-tooltip="Marcar como completada">
    ✓
</button>
```

Paso B: Crea el tooltip con CSS puro

```
/* Necesario: el botón debe tener position relative */
.task-card__btn {
    position: relative;
}

/* El tooltip: un pseudo-elemento invisible por defecto */
.task-card__btn[data-tooltip]::after {
    /* Lee el texto directamente del atributo HTML */
    content: attr(data-tooltip);

    /* Posición: sobre el botón */
    position: absolute;
    bottom: calc(100% + 8px);
```

```
left: 50%;
transform: translateX(-50%);

/* Estilos visuales */
background: var(--color-text);
color: var(--color-surface);
padding: 5px 10px;
border-radius: 6px;
font-size: 11px;
white-space: nowrap;

/* Invisible por defecto */
opacity: 0;
pointer-events: none;
transition: opacity 0.2s;
}

/* La flechita del tooltip */
.task-card__btn[data-tooltip]::before {
  content: "";
  position: absolute;
  bottom: calc(100% + 3px);
  left: 50%;
  transform: translateX(-50%);
  border: 5px solid transparent;
  border-top-color: var(--color-text);
  opacity: 0;
  transition: opacity 0.2s;
}

/* Al hacer hover: mostrar con fade */
.task-card__btn[data-tooltip]:hover::after,
.task-card__btn[data-tooltip]:hover::before {
  opacity: 1;
}
```

¿Por qué attr() es tan poderosa?

La función CSS `attr(data-tooltip)` lee en tiempo real el valor del atributo HTML. Esto significa que puedes tener 50 botones diferentes con 50 textos de tooltip distintos usando **exactamente el mismo CSS**. No necesitas crear una clase CSS por cada tooltip.

Reto adicional del Bonus Track

Si ya implementaste el tooltip básico, intenta agregar estas mejoras con CSS solamente:

- Tooltip con retraso (delay): el tooltip aparece 0.5s después del hover, no instantáneamente. Pista: transition-delay.
- Tooltip adaptativo: si el botón está en el lado izquierdo de la pantalla, el tooltip aparece hacia la derecha; si está a la derecha, hacia la izquierda. Pista: :first-child y :last-child selector.
- Tooltip en modo oscuro: el tooltip debe cambiar de color automáticamente cuando se activa el modo oscuro, usando las variables CSS que ya definiste.

Reflexión final: ¿Cuándo NO usarías CSS puro para los tooltips y preferirías JavaScript? Piensa en casos como: tooltips que se posicionan automáticamente cuando están cerca del borde de la pantalla, o tooltips con contenido HTML rico (imágenes, botones dentro del tooltip). Discute con tu equipo.

6. Checklist de Entrega

Antes de entregar tu proyecto, verifica que has completado todos estos puntos. Marca cada uno al revisarlo:

index.html

- El documento tiene `<!DOCTYPE html>` y `lang="es"`
- Se usa `<aside>` para el sidebar, `<header>` para la cabecera, `<main>` para el contenido principal
- Cada tarjeta de tarea es un `<article>` dentro de una `<section>`
- Los títulos siguen jerarquía correcta: `h1 > h2 > h3`
- Cada botón de acción tiene `data-id` con un valor único
- Los botones tienen clases BEM específicas: `--completar`, `--editar`, `--eliminar`
- El toggle de modo oscuro tiene un `id` para ser capturado por JS

css/style.css

- Las Custom Properties están definidas en `:root`
- El selector `[data-theme="dark"]` sobrescribe los colores
- El layout principal usa `display: grid` con `grid-template-areas`
- El sidebar tiene `position: sticky` y `height: 100vh`
- Las tarjetas usan `display: flex` con `gap` para separar elementos
- Las estadísticas usan `display: flex` con `flex-wrap: wrap`
- La Media Query cambia el layout en `max-width: 768px`
- En mobile, el sidebar tiene `position: fixed` y `bottom: 0`
- Todas las clases CSS siguen la metodología BEM

Mapa de Datos

- Has identificado al menos 8 propiedades de la entidad Tarea
- Cada propiedad tiene tipo de dato definido
- Los Enums tienen sus valores posibles listados
- Has escrito un objeto JavaScript de ejemplo con todas las propiedades

Revisión de calidad

- El dashboard se ve correcto en pantallas de 1280px de ancho
- El dashboard se ve correcto en pantallas de 375px de ancho (iPhone)
- El modo oscuro funciona al hacer clic en el toggle
- Las tarjetas muestran claramente la prioridad, fecha y responsable
- El código CSS está comentado en las secciones principales

¡Felicidades! Al completar este taller, tienes una base de maquetación profesional que será el punto de partida del próximo módulo de JavaScript. Cada `data-id` que pusiste en tus botones, y

cada clase BEM que escribiste, será capturada por el DOM. Tu maqueta no es solo estética: **es la interfaz de tu futura aplicación.**